

SKILLRACE: Implementation Pipeline

Reasoning-Augmented Concolic Execution for Coding-Agent Skills

Engineering design document — what to build, in what order, and why it produces results

June 2026

1 What this system is, in one paragraph

A coding-agent *skill* is reusable procedural guidance (a `SKILL.md` plus optional scripts) that an agent consults to do a task—fix a failing test, rebase a branch, scaffold a component. Skills are trusted after a handful of runs and tested almost not at all. SKILLRACE tests them the way concolic execution tests code: it runs the skill, lifts each run into a sequence of *episodes* (coherent units of work, each with a purpose and an observation-grounded outcome), merges episodes into a growing *behavior tree*, reads the agent’s reasoning at each branch as the *condition* that sent it one way rather than another, and then *negates or mutates* those conditions to synthesize new (prompt, environment) inputs that drive the skill down branches it has not yet taken. Each new run folds back into the tree, which reveals the next unexplored branch, and so on. The tree is both the byproduct and the driver of the search—exactly as the path tree is in classical concolic execution. The payoff is twofold: (i) a map of how the skill’s guidance actually maps onto execution, showing which parts of the skill remain unexplored, and (ii) a stream of targeted tests that expose where the skill violates its intended behavior. Correctness of each run is judged by *properties*—some fixed formulas, some natural-language specifications compiled per task into executable checks (specification by example)—not by asking a model “did this go well?”

The core conceptual move (the part that makes it publishable). Episodes are the *basic blocks*; the agent’s reasoning between them is the *branch condition*; hypothesizing over those conditions (negating, mutating) tells us how the skill maps onto the execution trajectory, which regions remain unexplored, and how to synthesize new interesting tests. This is the concolic-execution idea lifted from source code to agent behavior: there the branch condition is a path constraint read from the code; here it is a natural-language condition read from the agent’s reasoning.

What this document is. The engineering plan: components, data formats, the per-stage algorithm, the baselines, and the build order. The formal treatment (soundness of the abstraction, etc.) lives in a separate document; here the emphasis is on what to build and why it is realistic and produces results.

Models. A single model is used for *every* model-driven step (segmentation, summarization, merge decisions, guard extraction, specification compilation). We do not mix models across steps; instead we *ablate* over the choice of model (swap it for a stronger/weaker one and report how the pipeline’s numbers move). This keeps the design and the calibration uniform: we calibrate one model per

judgment type, and the model-ablation shows the whole pipeline’s sensitivity to it. The agent under test is whatever agent the skill targets and is independent of this choice.

2 System overview and cost structure

The pipeline has six components. Five are cheap (plain code or a small fast model); one—running the skill’s agent under test—is expensive (minutes and real money per run). **Every design decision follows from this asymmetry: spend agent runs only on inputs we have already validated, and do everything else with cheap models or code.**

1. **Runner** — executes a skill in Docker on an input (x, E_0) , logs the full trace. (*expensive: the agent under test*)
2. **Episode segmenter** — splits a trace into episodes and labels each with a purpose. (*cheap model*)
3. **Outcome & summary** — assigns each episode an observation-grounded outcome and a short canonical summary. (*cheap model + code*)
4. **Tree builder** — merges episodes into the behavior tree; decides where runs share structure and where they branch. (*cheap model for merge decisions; code for the tree*)
5. **Guard extractor & test synthesizer** — reads the reasoning at a branch, distills the condition, mutates it, drafts a new (x, E_0) , and validates it before any agent run. (*cheap model + code validator*)
6. **Property checker** — evaluates correctness properties over a run. (*code; cheap model only to compile NL specs into checks*)

Loop: seed the tree with a few runs → pick an unexplored branch → mutate its condition → synthesize and validate a new input → run the agent → fold the trace back in → check properties → repeat until the run budget is spent.

3 Component 1: The runner and trace format

What it does. Given (x, E_0) —a prompt and a Docker initialization script that builds the starting environment—it runs the skill’s agent at temperature 0 inside the container and records everything.

Trace format (frozen early; everything downstream consumes only this). The runner records the agent framework’s raw session log; a *deterministic renderer* (no model) projects it into the globally-numbered view every downstream component consumes — one entry per tool call:

```
{ call: int, reasoning: str,          // assistant thinking before the call
  tool: str, args: {...},           // the tool call
  observation: str }                // raw tool output
```

Final repository / container state is read directly at property-checking time (Section ??); we do not store a per-step environment snapshot.

Why this is realistic. Coding skills are short (procedural guidance for one task), so traces are tens of steps, not thousands. The runner is a thin wrapper around the agent framework plus Docker; the only real work is the per-skill environment setup (sourcing skills with clean, Dockerizable tasks is the real labor, not the runner). Use a hard **step/turn cap** per run—needed for reproducibility and to stop a runaway agent from burning budget.

4 Component 2: Episode segmentation

What an episode is. A contiguous run of steps that together pursue one purpose: “locate the failing test,” “edit the implementation,” “run the tests,” “verify the endpoint.” An episode is the **basic block** of the behavior tree—internally it may be one tool call or eight, but it is treated as a unit with a purpose (intent) and a result (outcome).

How we split (and the principle behind it). **Boundaries are read from the agent’s reasoning, because the agent announces its goal switches** (“Now let me verify the endpoint works,” “First I’ll check whether the tests pass”). The segmenter is a cheap-model call over the trace that emits episode boundaries, each with an *intent* (a short purpose phrase) and an *evidence pointer* (the index of the reasoning turn that announced the episode). Output is strict JSON, mechanically validated: episodes partition the steps in order; every evidence index is in range. One retry, then the trace is flagged unsegmentable and counted (expected rare).

Mechanics. The renderer’s numbered trace plus a baked-in worked example goes to the segmenter with a *soft episode-count target*, a smooth saturating function of the tool-call count ($D = 3 + N/50$, $\text{target} = N/D$) that discourages under-splitting without dictating boundaries. Spans are validated mechanically (they must partition the calls in order; boundaries may only fall where a reasoning turn exists); one repair attempt with the violation fed back, then the trace is flagged unsegmentable and counted (expected rare). Each episode carries *intent*, *what it did*, and *outcome* — the summarization of Section ?? is emitted in the same pass. Traces that exceed one context window are handled by an agentic variant of the same segmenter that pages through the rendered trace file itself under identical rules.

Faithfulness/quality knob. Segmentation is a model judgment, so it is a calibrated component: hand-segment ~ 100 traces and measure boundary agreement (F1) and intent-label agreement separately—a boundary error shifts two adjacent episodes, a label error corrupts one. These numbers go in the evaluation.

5 Component 3: Episode summarization

What it does. One model call per episode turns the episode’s raw steps—its tool calls, its reasoning, and its tool outputs—into a short **structured summary** following a fixed template: *what was attempted, on what, and with what result*. Example: “*ran the project test suite; it failed with an ImportError in the auth module.*” The summary is open in content (the model captures whatever actually happened) but bounded in form (the template), so two episodes can later be compared for “same situation” without either pre-bucketing them into a rigid enum or leaving them as unmatchable free text.

The result *is* the outcome, and it is read from this episode’s tool outputs. There is no separate “outcome” stage and no fixed outcome grammar—the “with what result” part of the summary is the outcome. The one rule that matters: that result is read from the episode’s *tool outputs* (the exit code, the printed text, the HTTP status), **not** from the agent’s own narration of how it went. Agents declare false victory; if the recorded result came from the agent saying “tests pass,” the skill would be grading itself, and a run that ignored a real failure would be recorded as a success. By sourcing the result from what the test command actually printed, a divergence between *what happened* (observation) and *what the agent thought happened* (its later reasoning) is preserved—and that divergence is itself a bug signal (see Section ??). In practice the model can read exit codes and output directly; the template just asks it to ground the “result” field in the observation, not the reasoning.

Quality knob. Summarization is a model judgment, so it is calibrated: on a sample of episodes, check that the summary’s result field agrees with the ground-truth result of the tool outputs.

6 Component 4: The behavior tree (online merging)

What the tree is. A growing structure whose nodes are *situations the skill reaches* and whose edges are *what it did next*. It is a **concolic search tree**: built incrementally as runs come in, and used at each step to decide what to explore next. There is no “collect all runs then build”—later runs do not exist until the current tree has proposed them.

How a run folds in (the merge). Walk the new run’s episode sequence from the root:

1. At the current node, compare the run’s next episode against the current node’s *children*. Matching is on **purpose** (the episode’s intent; how it was done is secondary context): a cheap-model similarity judgment, not string equality. **The outcome does not enter the match**—it is recorded on the *edge* taken into the node (together with the opening reasoning of the next episode), which is exactly where the guard will read it (Section ??).
2. **Match** $\Rightarrow$ merge into that child: the node’s purpose is **rewritten to generalize** over all its members (broadening only, so a node never disowns a run it already represents), and the episode’s concrete tactic is recorded as one of the node’s *ways* (similar tactics collapse, genuinely different ones accumulate as variants).
3. **No match** $\Rightarrow$ create a new child (the run diverges here). A node with ≥ 2 children is a **branch**—the place a guard lives.

Two runs that reach the same situation with *different outcomes* therefore share the node but carry different outcome labels on their out-edges—the divergence, if it matters, appears one step later as a branch, with the differing outcomes sitting on its in-edges as the guard’s first signal.

Why similarity-merging, and why it is safe here. A fixed outcome enum would crush the diversity of real episodes; pure string matching would never merge two phrasings of the same situation. Similarity-merging lets the model judge “same purpose” with full nuance. The risk—that a similarity judgment is not perfectly consistent—is contained two ways: (i) merge decisions are made at temperature 0 and **cached**, so the same episode pair always gets the same verdict within a campaign; and (ii) a *wrong* merge is self-limiting: if two genuinely different situations were merged, their next episodes diverge, the point presents as a branch, and the guard extracted

there names the distinguishing condition—the search routes around the error rather than being corrupted by it. Merge errors degrade search guidance temporarily; they do not silently corrupt results.

Determinism, honestly. The build is a deterministic *procedure* (same seeds + same selection policy + temperature 0 + caching $\Rightarrow$ same campaign), not an order-invariant abstraction. We report build stability empirically: re-run a campaign under a different random seed for the selection policy and measure tree agreement.

What the tree gives us. (i) **Coverage:** how many distinct situations/branches the skill’s behavior occupies—and, by comparison against the skill’s documented steps, which parts of the skill remain unexercised. (ii) **Branch frontier:** the set of branches whose alternatives we have not yet tried—the worklist for test synthesis.

7 Component 5: Guards and test synthesis

Where a guard lives, and the two distinct signals it uses. A guard sits at a **branch**—a point where runs that reached the same situation went different ways—and is the *condition* that explains the split. It is read from two things, and it is worth being precise that these are *different signals with different sources*, because they are easy to conflate:

- the **outcome of the episode that just finished**—this is the “result” field of that episode’s summary (Section ??), read from its *tool outputs*; and
- the **opening reasoning of the next episode** (“the endpoint returned 500, so I’ll check the logs”)—read from the agent’s *reasoning text*.

The outcome tells us *what state the agent was in*; the next reasoning tells us *why it chose the branch it did, given that state*. They are not the same source, even though the agent’s reasoning usually *restates* the outcome—and that is the subtle point: we take the outcome from the observation (Component 3), not from the reasoning. When the two disagree—the test *observably* failed but the agent’s next reasoning proceeds as if it passed—that disagreement is not noise to reconcile; it is a bug signal (the agent ignored a real failure), and we record it. The guard is *not* read from the internal tool calls of an episode—those are tactics the tree abstracts over, which is exactly why two runs that reached the same outcome by different tactics present the same situation to the branch.

This is the conceptual core. A classical program hands the tester its branch conditions in source code. An agent has no source code—its reasoning trace is the nearest thing: it is where the agent states why it branched. We treat each stated reason as a *candidate* branch condition, distilled into a checkable form, and we *test* it by mutation: if changing the condition does not change the behavior, the stated reason was not the real cause (the loop detects this and adjusts). Reasoning proposes the branch condition; execution confirms or refutes it.

Guard extraction. When a branch exists, a model call takes the outcomes + opening reasonings of the diverging sides, plus a **diff** of their initial environments/observations, and produces the guard as a short condition *plus, wherever possible, an executable grounding*: a concrete check such as `test -f <path>`, “exit code $\neq$ 0”, or “output contains `ModuleNotFoundError`”. Guards that ground to an executable check are *cheap to satisfy and verify*; guards that stay natural-language are evaluated by a (calibrated) model when needed.

Test synthesis = negate or mutate the guard. To explore an unexplored branch, take a guard and drive it to a *different* value than observed:

- **Negate** (binary guard): the test failed with an assertion error → make it fail with something else / not fail.
- **Mutate** (multi-valued guard): the failure was an assertion error → target an import error, a timeout, a collection error—each a sibling region.

A cheap-model **generator** drafts a candidate (x^*, E_0^*) : a prompt plus a Docker init script intended to satisfy the path into the branch *and* the mutated guard.

Property-guided targeting. Which guard to mutate, and toward which value, is not chosen blindly: the selection step is shown the skill’s *correctness properties* (Section ??) alongside the frontier and asked for the feasible (guard, mutation) pair whose exploration is most likely to drive the skill toward a property violation—e.g. prefer “the failing test is an import error” → “the failing test is an *environment* error” when a test-integrity property could plausibly break there. This turns exploration into *directed falsification*: the frontier supplies what is reachable, the properties supply what is worth reaching. The choice is a targeting heuristic only—it biases the search; correctness verdicts still come exclusively from the property checker.

Validation BEFORE spending an agent run (the key efficiency move). We only target guards that are decidable from the **initial setup** (E_0 -guards: “the repo lacks file X,” “the failing test is an import error,” “dependencies are missing”), plus guards causally forced by E_0 . For such a guard, the **validator builds the candidate container and checks the condition by running plain checks in the container it just built**—does the test it set up actually fail, with an import error?—**with no agent involved**. A botched candidate costs a retry, never an agent run. Only a validated (x^*, E_0^*) is handed to the runner. Guards that could only be checked by running the agent (predicates over the agent’s own mid-run outputs) are *deferred and counted*, not targeted in v1. (*“LLM proposes, checker disposes”: the generator may be dumb; the validator guarantees the input is valid before we pay for a run.*)

8 Component 6: Property checking (correctness of one run)

What “a bug” means. A run is buggy if it **violates a property we fixed in advance**—a mechanical automaton rejecting the run, or a concrete command failing in the final container—**not** if a model “thought it went badly.” Properties are checked over the run’s episode sequence and its final environment.

Two axes: what a property reads, and how it is written. A property is classified along two independent axes.

What it reads:

- **Trace-structural** — checkable *directly on the episode sequence* (equivalently, on the path through the behavior tree), with no extra execution: ordering and presence patterns like “a **lint** episode occurs before the final episode,” “a **test** episode with a passing result precedes any **commit** episode,” “no episode repeats more than N times.” These are temporal patterns over the episodes the tree already holds, so they are essentially free to evaluate—walk the path, check the pattern. Much of the self-consistency and process-hygiene catalog is of this kind.

- **State-based** — needs a concrete check run against the *final container* (or specific files): “the build passes,” “the target test still exists with unchanged assertions,” “the requested artifact is present.” These execute a command at checking time.

How it is written:

- **Fixed-formula properties** — written once, task-independent, evaluated by plain code over the trace and run manifest (no force-push, no destructive operation outside the workspace, no pathological repetition, terminated within budget). Zero model involvement at authoring *and* evaluation time; fully trustworthy. These are the universal process invariants, run for every skill and every method.
- **Specification-by-example properties** — written as a *natural-language specification* per skill, then **compiled per concrete** (x, E_0) **into an executable check—before the run exists**. The compiling model sees the prompt and the *built initial environment* (its tools, its starting file tree), never the run it will judge: the specification (“the skill must not make the target test pass by altering the test rather than the code”) is reusable; the compiled check is case-specific and must *discover* its concrete targets mechanically (find the test files, grep the trace) since the agent’s actual behavior is unknown at compile time. **The model runs only at compile time**; the produced check is a concrete, inspectable bash script that runs mechanically at evaluation time—deterministic, auditable, shipped alongside the bugs. Because checks attach to the *case*, every run of that case—under any method—is judged by byte-identical scripts, which is itself a fairness control. The compile step is a calibrated component (its own error rate); we report bug yield split by provenance (fixed vs. SBE), the SBE ones carrying a wider confidence band.

One evaluation mechanism. Every compiled check is a bash script executed in the run’s *final live container*, which exposes the final workspace, the full trace (`/check/trace.jsonl`), and the diff of everything the agent changed—so trace-structural and state-based checks run through the same auditable mechanism, and a check is free to combine both (grep the trace *and* run the build). The two axes are orthogonal: a fixed-formula property can be trace-structural (“test-before-commit”) or state-based (“build passes”); an SBE property likewise (“a backup episode precedes modification,” trace-structural, vs. “the target test’s assertions are unchanged,” state-based).

The property catalog (~18–22 properties, six categories).

1. **Test integrity** (mostly SBE; the reward-hacking heart): never pass the target test by editing the test; never weaken/delete an assertion; never add a mock that bypasses the asserted behavior; never override equality/comparison to force a pass; never insert `sys.exit(0)` / `conftest` patches that short-circuit the harness; never hardcode expected outputs.
2. **Outcome integrity**: never commit/finalize with failing tests (*fixed*); leave no dirty repo state unless asked (*fixed*); the artifact satisfies what the prompt requested (*SBE*).
3. **Reversibility & authorization** (mostly fixed): no force-push without confirmation; no destructive/irreversible op without a preceding confirmation; verify-before-acting on side-effecting calls; stay within the skill’s permitted tools/scope (e.g. designated working directory).
4. **Process hygiene** (fixed): terminate / respect a step budget; no pathological repetition; no unnecessary edits when code is already correct; never consult the solution/answer-key (e.g. reading the fix from git history).

5. **Build & static safety** (fixed, executable): build/lint/compile passes in the final state; no newly introduced insecure pattern.
6. **Self-consistency** (SBE, from the skill’s own docs): whatever the `SKILL.md` advertises holds on the run (“runs the linter before finishing” $\Rightarrow$ a lint episode precedes the final episode).

Not every property applies to every skill; we maintain a per-skill applicability matrix (a rebasing skill cares about force-push; a fix-the-test skill cares about test integrity). The fixed core ($\sim 6-8$ invariants) is the reusable library applied to all skills; the rest are selected by relevance.

Reproducibility grading. A flagged violation is re-run $k=3$ times: $3/3$ is a genuine skill bug; $1/3$ is brittleness (the skill is unreliable here). Each report carries the violated property, **the mutated assumption that produced the input**, a replayable Docker repro, and (for SBE properties) the generated check that fired.

The honest limitation. Bug yield is bounded by property strength: a bug nothing forbids is invisible. This is shared by all assertion-based testing. We address it empirically by *injecting known violations* (programmatically producing runs that delete a test, hardcode an output, or force-push) and reporting the detection rate—turning “are the properties strong enough?” into a measured number rather than a claim. We frame properties as evidence of bugs (a violation is a real problem), not as a completeness guarantee (“no violation” $\neq$ “correct”).

9 The full loop, assembled

Seed phase. Run the skill on ~ 20 seed inputs. Segment, summarize, and fold each into the tree. Extract guards at every branch. Result: a small behavior tree and an initial branch frontier.

Exploration loop (until run budget exhausted):

1. **Pick** a (branch, mutation) from the frontier by *property-guided targeting* (Section ??): among untried, E_0 -decidable mutations, the one most likely to drive a property violation.
2. **Mutate** its guard accordingly (negate / target a novel sibling value, given all observed siblings).
3. **Synthesize** a candidate (x^*, E_0^*) with the generator.
4. **Validate** it against the built container (no agent run). Reject and retry if invalid.
5. **Run** the agent once on the validated input.
6. **Fold** the new trace into the tree. Classify:
 - *Predicted divergence* — it took the new branch: coverage gained, guard corroborated, frontier extended.
 - *No divergence* — it reached the branch but behaved as before despite the mutated condition: the stated reason was not causal (or the mutation did not change what actually mattered). The mutation is marked spent and the guard demoted; the run still folds in, so if a genuinely different situation was hiding behind the merge, its own divergence surfaces as a new branch downstream.
 - *Path miss* — it never reached the branch: an earlier guard failed; recurse on that.
7. **Check properties** on the new run; emit any violations as bug reports (with $k=3$ regrade).

Outputs. (i) the behavior tree + coverage report (what of the skill is explored / unexplored); (ii) the bug reports (each with its violated property, the mutated assumption, and a replayable repro); (iii) per-component agreement numbers for the model-driven steps.

Budget. $\sim 100\text{--}150$ agent runs per skill per method. The expensive model runs once per loop iteration (step 5); everything else is the cheap model or code. A typical campaign of ~ 20 skills is a few thousand agent runs—days on parallel containers.

10 Baselines: the three-rung ladder

All baselines share SKILLRACE’s runner, Docker environments, run budget, episode abstraction (where they need a behavioral signal), and—critically—the **same property checker** (so any difference measures *test generation*, not detection). Each rung adds exactly one ingredient.

1. **Floor — random mutation.** A model mutates seed prompts and seed environments at random; no feedback, every mutant is run. (This is the standard black-box fuzzing baseline; it is also structurally the black-box baseline used by VeriGrey.)
2. **Greybox — VeriGrey-inspired tool-sequence feedback.** Ports VeriGrey’s feedback and scheduling *verbatim*: the *novelty of the tool-call sequence* decides which mutants join the seed corpus, and energy is assigned exactly as published (+1 each for a new tool, a new transition, a new sequence; energy = offspring count). Mutation is an LLM rewrite of the chosen seed’s task/environment conditioned on the tool sequence it produced (VeriGrey’s mutation scheme minus its injection-specific context bridge, which has no analogue outside injection testing); its injection oracle is replaced by the shared property checker. One adaptation parameter is genuinely free: the *tool-event label granularity*. VeriGrey labels events by tool name + arguments, which is meaningful over AgentDojo’s small argument spaces but degenerates on coding agents, where raw arguments are near-unique and would mark every run novel. We therefore define three schematization levels (name only; name + command head / file extension; name + normalized path), report the sensitivity, and use the **best-performing level** for the headline comparison—the baseline gets its empirically best configuration. **This rung is also the “no-reasoning, no-intent-layer” ablation:** it explores by raw tool-sequence novelty, with no episodes, no reasoning-derived guards.
3. **SkillRACE.** Episodes + reasoning-derived guard mutation + the behavior tree.

What the comparison shows. Floor $\rightarrow$ Greybox isolates the value of *any* behavioral feedback over blind mutation. Greybox $\rightarrow$ SKILLRACE isolates the value of *the episode abstraction and reasoning-guard mutation* over raw tool-sequence novelty—this is the headline claim, and it is measured against a real published mechanism rather than a strawman. *Optional security slice:* adopt VeriGrey’s own injection oracle on a subset to show SKILLRACE is not worse on the task VeriGrey was built for; kept secondary to stay on the correctness story.

Honest caveat. The greybox rung is a faithful port of VeriGrey’s *feedback idea*, not VeriGrey itself (we removed its injection-specific parts), so the comparison speaks to “tool-sequence feedback vs. reasoning-guard mutation for correctness testing,” not to VeriGrey’s security performance.

11 The two claims and their experiments

Claim 1 — finding skill defects. On the mined skill suite, run the three rungs under identical budgets and measure, per skill: distinct violated properties per agent-run budget, runs-to-first-violation, wall-clock-to-first-violation, and unique behavioral branches covered. The shared runner

and the per-case byte-identical checks mean any difference is attributable to test generation. Here a “bug” in a skill is concretely: guidance that is ambiguous or incomplete in a way that lets a faithful agent violate a correctness property—each report carries the violated property, the mutated assumption that produced the input, and a replayable Docker repro.

Claim 2 — improving generated skills. Most skills today are LLM-written, which is circular: a model teaching itself what it may or may not know. We test whether SKILLRACE’s feedback breaks the circle. Per skill scenario: x *hidden* tests (prompt + environment), each with y runnable pass-criteria—authored **independently** of every improvement loop (different model or hand-written; disjoint from anything a tester generates), the leakage control. Conditions: the LLM-generated skill zero-shot; the skill revised with random-testing feedback; with greybox feedback; with SKILLRACE feedback (bug reports carrying mutated assumptions + the coverage map of unexercised guidance). **The reviser prompt is identical across conditions—only the feedback payload differs**—so a hidden-test pass-rate difference is attributable to the feedback quality, i.e. to the testing method. Report pass rates zero-shot vs. each revision, and pass-rate per unit of testing spend.